

## ✓ Predicting High School Graduation Rates in Washington State School Districts

### Team Members:

Amanda Guan

Aram Isikbay

Trevor Neri

Viswanath R.S.

## Step 1: Problem Definition

### Problem Statement

Our project answers a key question in two complementary ways:

1. **Can we identify school districts with high risk of low graduation rates?** (Classification approach)
2. **Can we predict actual graduation rates to understand the magnitude of the problem?** (Regression approach)

We learned that both approaches are necessary. Identifying high-risk districts helps education leaders know where to allocate intervention resources. Predicting specific graduation rates helps them understand the scale of the problem, which is crucial for planning and budget allocation. Using both goals together enables us to provide the right level of support to schools that need it most, making the educational system fairer for all students.

### Why This Problem Matters

- **Proactive Intervention:** Identify at-risk districts for timely resource allocation
- **Data-Driven Policy Making:** Enable evidence-based decision making
- **Equitable Education:** Ensure similar opportunities for all students
- **Resource Optimization:** Target limited educational resources where they can have the greatest impact

## Application Feasibility of The Model

Platforms such as [data.wa.gov](https://data.wa.gov) and the [U.S. Data Catalog](https://nces.ed.gov/ipeds/datacenter/datacatalog/) store and maintain key education indicators, including student-teacher ratios, standardized test performance, resource allocation patterns, and historical graduation outcomes. These datasets make it possible to build predictive models that can be applied repeatedly as new data becomes available, ensuring a scalable and easily deployable solution for education leaders.

The feasibility of applying such a model is clear, but practical use still faces several challenges:

- **Scale:** Integrating and analyzing data from 295+ districts across multiple sources is time-consuming
- **Consistency:** Varying methodologies and assumptions can produce conflicting results
- **Bias:** Human interpretation can inadvertently influence conclusions

## Success Evaluation Criteria

To keep our project manageable and ensure useful results, we set high standards for model performance:

### Regression Model (Ridge):

- $R^2 \geq 0.6$  - Model should explain at least 60% of graduation rate variance
- $RMSE \leq 5$  percentage points - Predictions should be within 5 percentage points on average

### Classification Model (LightGBM):

- **Recall  $\geq 80\%$  for high-risk districts** - Our most important goal is to find at least 80% of truly at-risk schools
- **False positive rate  $< 30\%$**  - We believe it's much better to double-check a school that's doing okay than to miss a school that is struggling

### Model Interpretability:

- Clear identification of top predictive factors
- Actionable insights for decision-makers

## Scope and Constraints

### Scope:

- Washington State public school districts only
- 2022–2023 academic year dataset (most recent complete dataset)

**Constraints:**

- Public OSPI data only
- Excludes qualitative factors (teaching quality, community support, etc.)
- Cross-sectional, not longitudinal analysis
- Limited to available district-level indicators

## ✓ Step 2: Data Identification and Exploration

### Data Source Identification

#### 1. Graduation Rate Data (Target Variable)

- **Source:** WA OSPI Report Card - Graduation Data 2022-23
- **File:** Report\_Card\_Graduation\_2022-23\_20250721.csv
- **Content:** 4-year, 5-year, 6-year, and 7-year graduation rates
- **Granularity:** School, district, and demographic subgroup levels
- **URL:** <https://ospi.k12.wa.us/data-reporting/data-portal>

#### 2. Student-Teacher Ratio Data (Predictor Variable)

- **Source:** Washington Open Data Portal
- **File:** Report\_Card\_Student\_Teacher\_Ratio\_School\_Years\_2017-18\_to\_2023-24.csv
- **Content:** Student-teacher ratios across multiple school years
- **Granularity:** School and district levels
- **URL:** <https://catalog.data.gov/dataset/report-card-student-teacher-ratio-school-years-2017-18-to-2023-24-61974>

#### 3. Assessment Performance Data (Predictor Variable)

- **Source:** Washington State Open Data

- **File:** Report\_Card\_Assessment\_Data\_2022-23\_School\_Year\_20250712.csv
- **Content:** Standardized test results across proficiency levels
- **Granularity:** School, district, and grade levels
- **URL:** [https://data.wa.gov/education/Report-Card-Assessment-Data-2022-23-School-Year/xh7m-utwp/about\\_data](https://data.wa.gov/education/Report-Card-Assessment-Data-2022-23-School-Year/xh7m-utwp/about_data)

## Data Availability and Ethical Considerations

### Availability Assessment

All datasets are publicly accessible, free to download, and comply with FERPA. Data is provided at the district or school level and does not contain personally identifiable information. Coverage includes all public school districts in Washington State, ensuring comprehensive analysis for the most recent academic year.

### Potential Limitations

- Small district data may be suppressed to protect student privacy
- Some districts may have incomplete reporting
- Data represents outcomes, not causal relationships

## ✓ Data Exploration and Quality Assessment

To evaluate the suitability of our identified datasets, we conduct a preliminary exploration of each source:

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
```

```
try:
    grad_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Graduation_2019.csv')
    print(f"Graduation data loaded: {grad_df.shape}")
except FileNotFoundError:
    print("Graduation data file not found")
    grad_df = None

try:
    fund_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_1003_Funds_2019.csv')
    print(f"Funding data loaded: {fund_df.shape}")
except FileNotFoundError:
    print("Funding data file not found")
    fund_df = None

try:
    ratio_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Student_Teacher_Ratio_2019.csv')
    print(f"Student-teacher ratio data loaded: {ratio_df.shape}")
except FileNotFoundError:
    print("Student-teacher ratio data file not found")
    ratio_df = None

try:
    assess_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Assessment_Data_2019.csv')
    print(f"Assessment data loaded: {assess_df.shape}")
except FileNotFoundError:
    print("Assessment data file not found")
    assess_df = None
```

Graduation data loaded: (84004, 31)

Funding data loaded: (196, 21)

Student-teacher ratio data loaded: (19938, 20)

/tmp/ipython-input-1288119355.py:23: DtypeWarning: Columns (11) have mixed types. Specify dtype option on

assess\_df = pd.read\_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report\_Card\_Assessment\_Data\_2019.csv')
Assessment data loaded: (782445, 30)

## Dataset 1: Graduation Rate Data Quality Assessment

```

if grad_df is not None:
    print("=== GRADUATION DATA EXPLORATION ===")
    print(f"Dataset shape: {grad_df.shape}")
    print(f"\nColumn names: {list(grad_df.columns)[:10]}...")

    if 'OrganizationLevel' in grad_df.columns:
        org_levels = grad_df['OrganizationLevel'].value_counts()
        print(f"\nOrganization levels available: {dict(org_levels)}")

        district_grad = grad_df[grad_df['OrganizationLevel'] == 'District']
        print(f"District-level records: {len(district_grad)}")

        if 'Cohort' in district_grad.columns:
            cohorts = district_grad['Cohort'].value_counts()
            print(f"\nCohort types: {dict(cohorts)}")

            four_year = district_grad[district_grad['Cohort'] == 'Four Year']
            if 'StudentGroup' in four_year.columns:
                groups = four_year['StudentGroup'].value_counts()
                print(f"\nStudent groups: {list(groups.index)[:5]}...")

                all_students = four_year[four_year['StudentGroup'] == 'All Students']
                print(f"\nDistricts with 'All Students' 4-year graduation data: {len(all_students)}")

                if 'GraduationRate' in all_students.columns:
                    grad_rates = pd.to_numeric(all_students['GraduationRate'], errors='coerce')
                    valid_rates = grad_rates.dropna()
                    print(f"Valid graduation rates: {len(valid_rates)}")
                    print(f"Graduation rate range: {valid_rates.min():.1f}% - {valid_rates.max():.1f}%")
                    print(f"Average graduation rate: {valid_rates.mean():.1f}%")

            print("\nGraduation data appears suitable for analysis")
        else:
            print("Cannot assess graduation data quality")

```

```
=== GRADUATION DATA EXPLORATION ===
```

```
Dataset shape: (84004, 31)
```

```
Column names: ['SchoolYear', 'OrganizationLevel', 'County', 'ESDName', 'ESDOrganizationId', 'DistrictCode']
```

```
Organization levels available: {'School': np.int64(57405), 'District': np.int64(21746), 'County': np.int64(3)}
District-level records: 21746
```

```
Cohort types: {'Five Year': np.int64(5501), 'Four Year': np.int64(5466), 'Six Year': np.int64(5411), 'Seven Year': np.int64(5411)}
```

```
Student groups: ['Female', 'Non-Homeless', 'Low-Income', 'Non Migrant', 'Students without Disabilities']
```

```
Districts with 'All Students' 4-year graduation data: 262
```

```
Valid graduation rates: 196
```

```
Graduation rate range: 0.5% - 1.0%
```

```
Average graduation rate: 0.9%
```

```
Graduation data appears suitable for analysis
```

## ✓ Dataset 2: Student-Teacher Ratio Data Quality Assessment

```
if ratio_df is not None:
    print("=== STUDENT-TEACHER RATIO DATA EXPLORATION ===")
    print(f"Dataset shape: {ratio_df.shape}")
    print(f"\nColumn names: {list(ratio_df.columns)}")

    if 'SchoolYear' in ratio_df.columns:
        years = ratio_df['SchoolYear'].value_counts().sort_index()
        print(f"\nSchool years available: {list(years.index)}")

        recent_data = ratio_df[ratio_df['SchoolYear'] == '2022-23']
        print(f"2022-23 records: {len(recent_data)}")

        if 'OrganizationLevel' in recent_data.columns:
            org_levels = recent_data['OrganizationLevel'].value_counts()
            print(f"\nOrganization levels in 2022-23: {dict(org_levels)}")
```

```

if 'StudentToTeacherRatio' in recent_data.columns:
    ratios = pd.to_numeric(recent_data['StudentToTeacherRatio'], errors='coerce')
    valid_ratios = ratios.dropna()
    print(f"\nValid student-teacher ratios: {len(valid_ratios)}")
    if len(valid_ratios) > 0:
        print(f"Ratio range: {valid_ratios.min():.1f} - {valid_ratios.max():.1f}")
        print(f"Average ratio: {valid_ratios.mean():.1f}:1")

    print("\n Student-teacher ratio data appears suitable for analysis")
else:
    print("Cannot assess student-teacher ratio data quality")

```

=== STUDENT-TEACHER RATIO DATA EXPLORATION ===

Dataset shape: (19938, 20)

Column names: ['RowId', 'SchoolYear', 'OrganizationLevelId', 'OrganizationLevel', 'OrganizationId', 'Organ

School years available: ['2017-18', '2018-19', '2019-20', '2020-21', '2021-22', '2022-23', '2023-24']  
 2022-23 records: 2905

Organization levels in 2022-23: {'School': np.int64(2557), 'LEA': np.int64(336), 'ESD': np.int64(11), 'Sta

Valid student-teacher ratios: 2654

Ratio range: 0.0 - 234.0

Average ratio: 14.0:1

Student-teacher ratio data appears suitable for analysis

## ✓ Dataset 3: Assessment Performance Data Quality Assessment

```

if assess_df is not None:
    print("=== ASSESSMENT DATA EXPLORATION ===")
    print(f"Dataset shape: {assess_df.shape}")

    level_cols = [col for col in assess_df.columns if 'level' in col.lower()]
    print(f"\nProficiency level columns found: {level_cols[:5]}...")

```



```

if 'OrganizationLevel' in assess_df.columns:
    district_assess = assess_df[assess_df['OrganizationLevel'] == 'District']
    print(f"\nDistrict-level assessment records: {len(district_assess)}")

    if 'GradeLevel' in district_assess.columns:
        grades = district_assess['GradeLevel'].value_counts()
        print(f"\nGrade levels: {list(grades.index)[:8]}...")

        hs_grades = ['9', '10', '11', '12', 'All Grades']
        hs_data = district_assess[district_assess['GradeLevel'].astype(str).isin(hs_grades)]
        print(f"High school grade records: {len(hs_data)}")

        if len(hs_data) > 0 and level_cols:
            sample_col = level_cols[0] if level_cols else None
            if sample_col:
                prof_data = pd.to_numeric(hs_data[sample_col], errors='coerce')
                valid_prof = prof_data.dropna()
                print(f"\nSample proficiency data ({sample_col}):")
                print(f"Valid records: {len(valid_prof)}")
                if len(valid_prof) > 0:
                    print(f"Range: {valid_prof.min():.3f} - {valid_prof.max():.3f}")

            print("\n Assessment data appears suitable for analysis")
        else:
            print("Cannot assess assessment data quality")

```

=== ASSESSMENT DATA EXPLORATION ===

Dataset shape: (782445, 30)

Proficiency level columns found: ['OrganizationLevel', 'GradeLevel', 'Count Consistent Grade Level Knowledge']

District-level assessment records: 165153

Grade levels: ['All Grades', '05', '08', '06', '04', '03', '07', '10']...

High school grade records: 53181

Sample proficiency data (OrganizationLevel):

Valid records: 0

Assessment data appears suitable for analysis

## ✓ Dataset Integration Feasibility Check

```
print("=== DATA INTEGRATION FEASIBILITY ===")

datasets = {'Graduation': grad_df, 'Funding': fund_df, 'Ratio': ratio_df, 'Assessment': assess_df}

district_cols = {}
for name, df in datasets.items():
    if df is not None:
        potential_cols = [col for col in df.columns if 'district' in col.lower() or 'lea' in col.lower()]
        if potential_cols:
            district_cols[name] = potential_cols[0]
            print(f" {name}: District column '{potential_cols[0]}' found")
        else:
            print(f" {name}: No obvious district column found")
            district_cols[name] = None

if district_cols:
    print("\n=== SAMPLE DISTRICT NAMES FOR CONSISTENCY CHECK ===")
    for name, col in district_cols.items():
        if col and datasets[name] is not None:
            sample_districts = datasets[name][col].dropna().unique()[:5]
            print(f"{name}: {list(sample_districts)}")

print("\n Data integration appears feasible with proper name standardization")
```

```
=== DATA INTEGRATION FEASIBILITY ===
Graduation: District column 'DistrictCode' found
Funding: District column 'District Code' found
Ratio: District column 'LEAOrganizationId' found
Assessment: District column 'DistrictCode' found

=== SAMPLE DISTRICT NAMES FOR CONSISTENCY CHECK ===
```

```
Graduation: [np.float64(14005.0), np.float64(27901.0), np.float64(21226.0), np.float64(29103.0), np.float64(22017.0)]
Funding: [np.int64(1147), np.int64(3017), np.int64(3052), np.int64(3053), np.int64(3400)]
Ratio: [np.float64(100010.0), np.float64(100011.0), np.float64(100012.0), np.float64(100013.0), np.float64(100014.0)]
Assessment: [np.float64(32801.0), np.float64(14005.0), np.float64(21226.0), np.float64(22017.0), np.float64(27901.0)]
```

Data integration appears feasible with proper name standardization

## Data Suitability and Project Feasibility

The selected datasets provide strong coverage of Washington State public school districts from the 2022-'23 academic year, and are sourced directly from official state education authorities. District-level granularity matches our analytical needs, and graduation rates provide clear, quantifiable success metrics.

## Limitations and Mitigation

Some small districts have suppressed data to protect privacy, and not all districts report complete information across all measures. The financial data focuses on federal intervention grants rather than general per-pupil expenditures. To address these issues, we will standardize district names, handle missing values appropriately, derive meaningful metrics from raw data, validate findings against known research, and conduct sensitivity analysis with different inclusion criteria.

## Feasibility of Approach

All datasets are publicly accessible and sufficiently detailed for machine learning. The dataset scale (~295 districts) is appropriate for standard regression and tree-based techniques, which are explainable and well suited to this problem. Established educational indicators such as student-teacher ratios and assessment performance provide meaningful predictors, ensuring both technical feasibility and actionable results for informing education policy.

## Resources and Timeline

The data can be processed using standard Python workflows. Model development and validation (including cross-validation and holdout testing) are straightforward to implement, and results can be visualized.



## Step 3: Formal Plan to Evaluate Model Efficacy

### ✓ Evaluation Proposal #1: Risk-Based Regression Metrics (Lead: Amanda)

#### Primary Evaluation Metrics

##### Graduation Rate Risk Classification

- **High Risk:** Districts with graduation rates < 25th percentile (bottom quartile)
- **Moderate Risk:** Districts with graduation rates between 25th-75th percentiles
- **Low Risk:** Districts with graduation rates > 75th percentile (top quartile)

#### Why These Metrics Are Appropriate

Risk-based classification mirrors real-world educational policy decisions. While federal ESSA guidelines use a fixed 67% threshold, our percentile-based approach offers several advantages:

##### Comparison with ESSA Guidelines:

- **Federal (ESSA) Approach:** Fixed 67% graduation rate threshold
- **Washington State:** Applies the 67% threshold using three-year composite rates
- **Our Approach:** Dynamic percentile-based thresholds (25th/75th percentiles)

##### Why Our Approach Is Better:

- **Data-Driven Flexibility:** Thresholds adapt to actual performance distribution
- **Relative Performance:** Identifies districts performing poorly relative to their peers
- **Balanced Distribution:** Ensures meaningful differentiation across performance levels
- **State-Specific Context:** Accounts for Washington's unique educational landscape
- **Predictive Focus:** Designed for forecasting future risk
- **Resource Optimization:** Creates more nuanced categories for targeted intervention

##### Additional Benefits:

- **Actionable:** Provides clear categories for resource allocation
  - **Interpretable:** Education officials can easily understand risk levels
  - **Policy-Relevant:** Aligns with state education department operations
  - **Statistically Sound:** Based on actual data distribution
- 

✓ Evaluation Proposal #2: Risk-Based Classification Metrics (Lead: Viswanath)

✓ Primary Evaluation Metrics - For Multi Class Classification (High/Medium/Low)

1. **Accuracy:** The proportion of correct predictions out of all predictions. While a good general indicator, it won't be our sole measure of success.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

2. **Precision:** Of all the districts the model predicted to be in a certain risk category, what percentage were actually in that category? High precision for the "High Risk" class means we can be confident in the districts our model flags for intervention.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

3. **Recall:** Of all the districts that were actually in a certain risk category, what percentage did our model correctly identify? High recall for "High Risk" is critical to ensure we don't miss districts that are genuinely at risk.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

4. **F1-Score:** The harmonic mean of precision and recall. This provides a balanced measure that is especially useful when precision and recall have different importance. We will use a weighted F1-score to account for class imbalance, where each class's F1-score is weighted by its size in the test set.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. **ROC-AUC** : The ROC (Receiver Operating Characteristic) curve is a graph that visualizes our model's ability to distinguish between classes. It plots the True Positive Rate (Recall) against the False Positive Rate at various classification thresholds.

### Success Criteria and Evaluation Method

- **Success Criteria:** The model will be considered successful if it achieves a **weighted F1-score of at least 0.70**. More importantly, it must have a **recall of at least 0.80 for the "High Risk" class**, ensuring we capture the vast majority of districts that need attention.
- **Evaluation Data and Method:** We will split our data into a training set (80%) and a holdout test set (20%). The holdout set will be used only once at the end to provide an unbiased final evaluation. During model development and hyperparameter tuning, we will use **k-fold cross-validation** on the training data. We will also use a **confusion matrix** to visualize model performance across the three classes and identify specific misclassification patterns.

### Scalability:

- **Throughput:** For our project, throughput is the rate at which our model can process school districts. We'd measure this in districts evaluated per minute. This is crucial because a state agency might need to re-evaluate all ~300 districts at once when new annual data is released. A model with high throughput can generate a fresh risk assessment for the entire state quickly, making it operationally efficient.
- **Latency** : It is the time it takes to generate a risk profile for a single school district. We'd measure this in milliseconds (ms).

```
district_baseline_df = grad_df[
    (grad_df['SchoolYear'] == '2022-23') &
    (grad_df['OrganizationLevel'] == 'District') &
    (grad_df['Cohort'] == 'Four Year') &
    (grad_df['StudentGroup'] == 'All Students')
]

grad_rates = pd.to_numeric(district_baseline_df['GraduationRate'], errors='coerce')

if grad_rates.max() <= 1.0:
```

```

grad_rates *= 100

valid_rates = grad_rates.dropna()

p25 = valid_rates.quantile(0.25)
p75 = valid_rates.quantile(0.75)
mean_rate = valid_rates.mean()

high_risk_threshold = p25
low_risk_threshold = p75

high_risk_count = (valid_rates < high_risk_threshold).sum()
moderate_risk_count = (
    (valid_rates >= high_risk_threshold) & (valid_rates <= low_risk_threshold)
).sum()
low_risk_count = (valid_rates > low_risk_threshold).sum()
total_valid_districts = len(valid_rates)

print("=== RISK LEVEL THRESHOLDS (based on 2022-23 data) ===")
print(f"25th percentile: {p25:.1f}%")
print(f"75th percentile: {p75:.1f}%")
print(f"Mean: {mean_rate:.1f}%")

print("\n=== PROPOSED RISK CLASSIFICATION ===")
print(f"HIGH RISK: Graduation rate < {high_risk_threshold:.1f}%")
print(f"MODERATE RISK: Graduation rate {high_risk_threshold:.1f}% - {low_risk_threshold:.1f}%")
print(f"LOW RISK: Graduation rate > {low_risk_threshold:.1f}%")

print("\n=== DISTRICT DISTRIBUTION BY RISK LEVEL ===")
print(f"High Risk: {high_risk_count} districts ({high_risk_count/total_valid_districts*100:.1f}%)")
print(f"Moderate Risk: {moderate_risk_count} districts ({moderate_risk_count/total_valid_districts*100:.1f}%)")
print(f"Low Risk: {low_risk_count} districts ({low_risk_count/total_valid_districts*100:.1f}%)")

=== RISK LEVEL THRESHOLDS (based on 2022-23 data) ===
25th percentile: 83.0%
75th percentile: 91.1%
Mean: 86.1%

=== PROPOSED RISK CLASSIFICATION ===

```

```
HIGH RISK: Graduation rate < 83.0%  
MODERATE RISK: Graduation rate 83.0% - 91.1%  
LOW RISK: Graduation rate > 91.1%
```

```
=== DISTRICT DISTRIBUTION BY RISK LEVEL ===  
High Risk: 49 districts (25.0%)  
Moderate Risk: 98 districts (50.0%)  
Low Risk: 49 districts (25.0%)
```

## ✓ Step 4: Modeling Technique Proposal

### Modeling Proposal #1: Risk-Aware Regression (Lead: Aram)

**Associated Evaluation Proposal:** Evaluation Proposal #1: Risk-Based Regression Metrics (Lead: Amanda)

#### 1. Rationale for a Simple, Interpretable Regression Approach

This proposal predicts each district's 4-year graduation rate percentage directly and then converts predictions into risk tiers (High/Moderate/Low) using the class p25/p75 thresholds from step 3. A regularized linear model keeps the system transparent, fast, and feasible for our team while preserving continuous fidelity for planning and benchmarking. Regression provides actionable magnitude (how far below target a district is) and supports the risk metrics from step 3 after thresholding.

#### 2. Modeling Technique: Regularized Linear Regression (Ridge)

- **Why it is appropriate:**
  - Continuous target with small/mediuim features
  - Collinearity-robust (assessment indicators often correlate)
  - Fast and reproducible: Easy to defend to non-technical stakeholders and district leaders
- **Tools:** `scikit-learn` (`Pipeline`, `SimpleImputer`, `StandardScaler`, `Ridge`, `GridSearchCV`)

#### 3. Interpretability Plan: Coefficients + Permutation Importance



- Standardized coefficients communicate direction/magnitude of effects (e.g., how assessment proficiency and student/teacher ratio relate to graduation rate)
- Scatterplot: Plot predicted vs actuals and a calibration line to show fit quality, alongside RMSE/R<sup>2</sup>

## Modeling Proposal #2 : High-Performance Classification (Lead: Trevor)

**Associated Evaluation Proposal:** Evaluation Proposal #2: Risk-Based Classification Metrics (Lead: Viswanath)

---

### 1. Rationale for a High-Performance Classification Approach

This proposal outlines an advanced modeling strategy to classify school districts as `Is_At_Risk` (Yes/No). In contrast to linear models that predict exact graduation rates, our approach focuses on predictive accuracy, providing a clear, actionable list of at-risk districts.

We will use a **Gradient Boosting Machine (GBM)**, a state-of-the-art ensemble technique renowned for its high performance on tabular data. This aligns directly with our goal of creating the most accurate predictive tool possible. While GBMs are often considered "black boxes," we will leverage **SHAP (SHapley Additive exPlanations)**, a leading-edge Explainable AI (XAI) technique, to interpret the model's predictions and understand the key drivers of risk.

This "accuracy-first" approach provides a strong competitive alternative to simpler, more interpretable models like linear regression.

---

### 2. Modeling Technique: Gradient Boosting Machine (LGBM)

- **Why it is appropriate:** Gradient Boosting models consistently win machine learning competitions for tabular data. They excel at uncovering complex, non-linear relationships and feature interactions that other models miss. For a problem like identifying at-risk school districts, this power is crucial for maximizing predictive accuracy. We will use the **LightGBM** implementation, which is known for its speed and efficiency.
- **Tools:** The `LGBMClassifier` from the `lightgbm` Python library.
- **Code Skeleton:**

```
import lightgbm as lgb
from sklearn.model_selection import train_test_split

# Assume 'X' is a DataFrame of features and 'y' is the 'Is_At_Risk' target series
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Calculate scale_pos_weight to handle class imbalance, a key for risk modeling
# This helps the model pay more attention to the minority "at-risk" class
scale_pos_weight = y_train.value_counts()[0] / y_train.value_counts()[1]

# Initialize the LightGBM Classifier
lgbm = lgb.LGBMClassifier(objective='binary',
                          random_state=42,
                          scale_pos_weight=scale_pos_weight)

# Train the model
lgbm.fit(X_train, y_train)

# The trained 'lgbm' model is now ready to make predictions on the test set,
# which will be fed into the evaluation metrics plan.
# y_pred = lgbm.predict(X_test)
# y_prob = lgbm.predict_proba(X_test)[:, 1]
```

### 3. Interpretability Plan: SHAP Values

To ensure our powerful model is not a complete "black box," we will use SHAP to explain its outputs. This allows us to maintain the high performance of Gradient Boosting while still providing deep insights into *why* the model makes its decisions.

- **Why it is appropriate:** SHAP is the industry-standard method for explaining complex tree-based models. It provides a mathematically sound way to decompose any single prediction into the contributions of each feature. This allows us to answer both global questions ("What features are most important overall?") and local questions ("Why was *this specific* district flagged as at-risk?").

- **Tools:** The `shap` Python library.
- **Code Skeleton & Output Interpretation:**

```
import shap

# Create a SHAP explainer object from our trained LightGBM model
explainer = shap.TreeExplainer(lgbm)

# Calculate SHAP values for the test set
shap_values = explainer.shap_values(X_test)

# Generate a summary plot to show global feature importance
# This plot shows which features have the biggest impact on predicting risk.
shap.summary_plot(shap_values[1], X_test, plot_type="bar")
```

This SHAP plot will serve as our primary tool for interpretation, allowing us to present a list of the top factors that contribute to a district being classified as "at-risk," directly connecting our high-performance model back to actionable policy insights.

## Proposed Task Assignments

To meet the project's requirements for exploring alternative approaches, our team of four will be split into two sub-groups. Each group will be responsible for developing one of the two competing proposals for evaluation and modeling.

### Sub-Group A:

- Evaluation Proposal #1 Lead (Step 3): **Amanda**
- Modeling Proposal #1 Lead (Step 4): **Aram**
- Implementation & Testing Task #1 Lead (Step 5): **Aram**

### Sub-Group B:

- Evaluation Proposal #2 Lead (Step 3): **Viswanath**
- Modeling Proposal #2 Lead (Step 4): **Trevor**
- Implementation & Testing Task #2 Lead (Step 5): **Trevor**

## ✓ Modeling Implementation #1: Risk-Aware Regression (Lead: Aram)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
from sklearn.inspection import permutation_importance
import warnings
warnings.filterwarnings('ignore')

# Data Preparation
print("Loading and preparing data...")

# Load graduation data
grad_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Graduation_2022-23')
grad_filtered = grad_df[
    (grad_df['OrganizationLevel'] == 'District') &
    (grad_df['StudentGroup'] == 'All Students') &
    (grad_df['Cohort'] == 'Four Year')
].copy()

district_data = grad_filtered[['DistrictName', 'GraduationRate']].copy()
district_data.dropna(subset=['GraduationRate'], inplace=True)
district_data['DistrictName'] = district_data['DistrictName'].str.lower().str.replace(' school district', '')

print(f"Loaded graduation data for {len(district_data)} districts")
```

```

# Load student-teacher ratio data
ratio_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Student_Teacher_
ratio_filtered = ratio_df[
    (ratio_df['SchoolYear'] == '2022-23') &
    (ratio_df['OrganizationLevel'] == 'LEA')
].copy()

ratio_data = ratio_filtered[['LEAName', 'StudentToTeacherRatio']].copy()
ratio_data.rename(columns={'LEAName': 'DistrictName'}, inplace=True)
ratio_data['DistrictName'] = ratio_data['DistrictName'].str.lower().str.replace(' school district', '').str.strip()

print(f"Loaded student-teacher ratio data for {len(ratio_data)} districts")

# Load 1003 funds data
funds_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_1003_Funds_2022-
funds_data = funds_df[['District Name']].copy() # Note: keeping the original column name with typo
funds_data.rename(columns={'District Name': 'DistrictName'}, inplace=True)
funds_data['Received_1003_Funds'] = 1
funds_data.drop_duplicates(inplace=True)
funds_data['DistrictName'] = funds_data['DistrictName'].str.lower().str.replace(' school district', '').str.strip()

print(f"Loaded 1003 funds data for {len(funds_data)} districts")

# Load assessment data
try:
    assessment_df = pd.read_csv('/content/drive/MyDrive/Grad-400-graduation-rate/data/Report_Card_Assessm
    assessment_filtered = assessment_df[
        (assessment_df['OrganizationLevel'] == 'District') &
        (assessment_df['StudentGroup'] == 'All Students')
    ].copy()

    performance_col = 'Percent Consistent Grade Level Knowledge And Above'

    # Clean percentage values
    assessment_filtered = assessment_filtered[
        (assessment_filtered[performance_col].notna()) &
        (assessment_filtered[performance_col] != 'N<10') &
        (assessment_filtered[performance_col] != 'NULL')

```

```

].copy()

assessment_filtered[performance_col] = assessment_filtered[performance_col].str.replace('%', '')
assessment_filtered[performance_col] = pd.to_numeric(assessment_filtered[performance_col], errors='co

# Aggregate by district and test subject
assessment_agg = assessment_filtered.pivot_table(
    index='DistrictName',
    columns='TestSubject',
    values=performance_col,
    aggfunc='mean'
).reset_index()

# Rename columns for clarity
assessment_agg.rename(columns={
    'ELA': 'ELA_Performance',
    'Math': 'Math_Performance',
    'Science': 'Science_Performance'
}, inplace=True)

assessment_agg['DistrictName'] = assessment_agg['DistrictName'].str.lower().str.replace(' school dist

print(f"Loaded assessment data for {len(assessment_agg)} districts")

except Exception as e:
    print(f"Warning: Assessment data could not be loaded ({e}). Creating placeholder columns.")
    assessment_agg = pd.DataFrame(columns=['DistrictName', 'ELA_Performance', 'Math_Performance', 'Scienc

# Merge All Prepared DataFrames
print("\nMerging datasets...")

final_df = pd.merge(district_data, ratio_data, on='DistrictName', how='left')
final_df = pd.merge(final_df, funds_data, on='DistrictName', how='left')
final_df['Received_1003_Funds'] = final_df['Received_1003_Funds'].fillna(0)

if len(assessment_agg) > 0:
    final_df = pd.merge(final_df, assessment_agg, on='DistrictName', how='left')
else:
    # Create placeholder assessment columns

```

```
final_df['ELA_Performance'] = np.nan
final_df['Math_Performance'] = np.nan
final_df['Science_Performance'] = np.nan

# Drop rows where essential features are missing
final_df.dropna(subset=['GraduationRate', 'StudentToTeacherRatio'], inplace=True)

print(f"Final merged dataset: {final_df.shape[0]} districts with {final_df.shape[1]} features")

# Create Risk Tiers Based on P25/P75 Thresholds
print("\nCreating risk tier thresholds...")

p25_threshold = final_df['GraduationRate'].quantile(0.25)
p75_threshold = final_df['GraduationRate'].quantile(0.75)

print(f"P25 threshold (High Risk): {p25_threshold:.3f}")
print(f"P75 threshold (Low Risk): {p75_threshold:.3f}")

def create_risk_tier(grad_rate):
    if grad_rate < p25_threshold:
        return 'High Risk'
    elif grad_rate < p75_threshold:
        return 'Moderate Risk'
    else:
        return 'Low Risk'

final_df['Risk_Tier_Actual'] = final_df['GraduationRate'].apply(create_risk_tier)

# Feature Engineering and Model Preparation
print("\nPreparing features for modeling...")

# Define feature columns
feature_columns = [
    'StudentToTeacherRatio',
    'Received_1003_Funds',
    'ELA_Performance',
    'Math_Performance',
    'Science_Performance'
]
```

```
# Check which assessment features are available
available_features = []
for col in feature_columns:
    if col in final_df.columns:
        available_features.append(col)
        non_null_count = final_df[col].notna().sum()
        print(f" {col}: {non_null_count}/{len(final_df)} districts have data")

print(f"\nUsing {len(available_features)} features: {available_features}")

X = final_df[available_features]
y = final_df['GraduationRate']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=final_df['Risk_Tier_Actual']
)

print(f"\nTraining set: {X_train.shape[0]} districts")
print(f"Test set: {X_test.shape[0]} districts")

# Ridge Regression Pipeline with Grid Search
print("\nTraining Ridge regression model...")

# Create preprocessing and modeling pipeline
pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('ridge', Ridge(random_state=42))
])

# Grid search for optimal alpha
param_grid = {
    'ridge__alpha': [0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
}

grid_search = GridSearchCV(
    pipeline,
    param_grid,
```



```
        cv=5,
        scoring='neg_mean_squared_error',
        n_jobs=-1
    )

    grid_search.fit(X_train, y_train)

    print(f"Best alpha: {grid_search.best_params_['ridge__alpha']}")
    print(f"Best CV score (RMSE): {np.sqrt(-grid_search.best_score_):.4f}")

    # Model Evaluation
    print("\n=== MODEL EVALUATION ===")

    # Make predictions
    y_pred_train = grid_search.predict(X_train)
    y_pred_test = grid_search.predict(X_test)

    # Calculate metrics
    train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))
    test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))
    train_r2 = r2_score(y_train, y_pred_train)
    test_r2 = r2_score(y_test, y_pred_test)
    train_mae = mean_absolute_error(y_train, y_pred_train)
    test_mae = mean_absolute_error(y_test, y_pred_test)

    print(f"Training RMSE: {train_rmse:.4f}")
    print(f"Test RMSE: {test_rmse:.4f}")
    print(f"Training R2: {train_r2:.4f}")
    print(f"Test R2: {test_r2:.4f}")
    print(f"Training MAE: {train_mae:.4f}")
    print(f"Test MAE: {test_mae:.4f}")

    # Risk Tier Classification Performance
    print("\n=== RISK TIER CLASSIFICATION ===")

    # Create risk tiers for predictions
    def create_risk_tier_from_prediction(pred_rate):
        if pred_rate < p25_threshold:
            return 'High Risk'
```

```
elif pred_rate < p75_threshold:
    return 'Moderate Risk'
else:
    return 'Low Risk'

y_pred_tiers = [create_risk_tier_from_prediction(pred) for pred in y_pred_test]
y_actual_tiers = [create_risk_tier(actual) for actual in y_test]

# Calculate tier classification accuracy
tier_accuracy = sum(1 for pred, actual in zip(y_pred_tiers, y_actual_tiers) if pred == actual) / len(y_pred_tiers)
print(f"Risk Tier Classification Accuracy: {tier_accuracy:.3f}")

# Tier distribution
tier_counts = pd.Series(y_actual_tiers).value_counts()
print("\nActual Risk Tier Distribution:")
for tier, count in tier_counts.items():
    print(f" {tier}: {count} districts ({count/len(y_actual_tiers)*100:.1f}%)")

# Feature Interpretability
print("\n=== FEATURE INTERPRETABILITY ===")

# Get the trained pipeline
best_pipeline = grid_search.best_estimator_

# Extract feature names after preprocessing
feature_names = available_features

# Get standardized coefficients
ridge_model = best_pipeline.named_steps['ridge']
coefficients = ridge_model.coef_

print("Standardized Ridge Coefficients:")
coef_df = pd.DataFrame({
    'Feature': feature_names,
    'Coefficient': coefficients,
    'Abs_Coefficient': np.abs(coefficients)
}).sort_values('Abs_Coefficient', ascending=False)

for _, row in coef_df.iterrows():
```

```
print(f" {row['Feature']}: {row['Coefficient']:.4f}")

# Permutation importance
print("\nPermutation Importance (Test Set):")
perm_importance = permutation_importance(
    best_pipeline, X_test, y_test,
    n_repeats=10, random_state=42
)

perm_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': perm_importance.importances_mean,
    'Std': perm_importance.importances_std
}).sort_values('Importance', ascending=False)

for _, row in perm_df.iterrows():
    print(f" {row['Feature']}: {row['Importance']:.4f} ( $\pm$ {row['Std']:.4f})")
```

Loading and preparing data...  
Loaded graduation data for 196 districts  
Loaded student-teacher ratio data for 336 districts  
Loaded 1003 funds data for 87 districts  
Loaded assessment data for 288 districts

Merging datasets...  
Final merged dataset: 196 districts with 8 features

Creating risk tier thresholds...  
P25 threshold (High Risk): 0.830  
P75 threshold (Low Risk): 0.911

Preparing features for modeling...  
StudentToTeacherRatio: 196/196 districts have data  
Received\_1003\_Funds: 196/196 districts have data  
ELA\_Performance: 194/196 districts have data  
Math\_Performance: 193/196 districts have data  
Science\_Performance: 193/196 districts have data

Using 5 features: ['StudentToTeacherRatio', 'Received\_1003\_Funds', 'ELA\_Performance', 'Math\_Performance', 'Science\_Performance']

Training set: 137 districts

```

Test set: 59 districts

Training Ridge regression model...
Best alpha: 10.0
Best CV score (RMSE): 0.0575

=== MODEL EVALUATION ===
Training RMSE: 0.0553
Test RMSE: 0.0640
Training R2: 0.2996
Test R2: 0.3222
Training MAE: 0.0408
Test MAE: 0.0461

=== RISK TIER CLASSIFICATION ===
Risk Tier Classification Accuracy: 0.559

Actual Risk Tier Distribution:
  Moderate Risk: 29 districts (49.2%)
  High Risk: 15 districts (25.4%)
  Low Risk: 15 districts (25.4%)

=== FEATURE INTERPRETABILITY ===
Standardized Ridge Coefficients:
  Math_Performance: 0.0202
  ELA_Performance: 0.0177
  StudentToTeacherRatio: 0.0175
  Science_Performance: -0.0135
  Received_1003_Funds: -0.0029

Permutation Importance (Test Set):
  Math_Performance: 0.3076 (±0.0482)
  ELA_Performance: 0.2336 (±0.0430)
  StudentToTeacherRatio: 0.0536 (±0.0301)
  Received_1003_Funds: 0.0110 (±0.0104)

```

## ✓ Modeling Evaluation #1: Risk-Aware Regression (Lead: Amanda)

```

# Summary report
print("\n" + "="*60)
print("RIDGE REGRESSION MODEL SUMMARY")
print("="*60)

print(f"\nDATASET:")
print(f"    Total districts: {len(final_df)}")
print(f"    Features used: {len(available_features)}")
print(f"    Train/Test split: {len(X_train)}/{len(X_test)}")

print(f"\nRISK TIER THRESHOLDS:")
print(f"    High Risk: < {p25_threshold:.3f} ({tier_counts.get('High Risk', 0)} districts)")
print(f"    Moderate Risk: {p25_threshold:.3f} - {p75_threshold:.3f} ({tier_counts.get('Moderate Risk', 0)} districts)")
print(f"    Low Risk: > {p75_threshold:.3f} ({tier_counts.get('Low Risk', 0)} districts)")

print(f"\nMODEL PERFORMANCE:")
print(f"    Test R²: {test_r2:.3f}")
print(f"    Test RMSE: {test_rmse:.3f}")
print(f"    Test MAE: {test_mae:.3f}")
print(f"    Risk Tier Accuracy: {tier_accuracy:.3f}")
print(f"    Optimal Alpha: {grid_search.best_params_['ridge__alpha']}")

print(f"\nTOP PREDICTIVE FEATURES:")
for i, (_, row) in enumerate(coef_df.head(3).iterrows()):
    direction = "increases" if row['Coefficient'] > 0 else "decreases"
    print(f"    {i+1}. {row['Feature']}: {direction} graduation rate")

print(f"\nMODEL INTERPRETATION:")
print(f"    The model explains {test_r2:.1%} of graduation rate variance")
print(f"    Average prediction error: {test_mae:.1%} graduation rate")
print(f"    Risk tier classification is {tier_accuracy:.1%} accurate")

print("\n" + "="*60)
print("Analysis complete. Visualizations saved as 'graduation_rate_ridge_analysis.png'")
print("="*60)

# Graphs
print("\nCreating visualizations...")

```

```
plt.style.use('default')
fig, ((ax1, ax2), (ax3, ax4)) = plt.subplots(2, 2, figsize=(15, 12))

# 1. Predicted vs Actual Scatterplot
ax1.scatter(y_test, y_pred_test, alpha=0.6, color='steelblue')
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Perfect Predict:
ax1.set_xlabel('Actual Graduation Rate')
ax1.set_ylabel('Predicted Graduation Rate')
ax1.set_title(f'Predicted vs Actual Graduation Rates\nTest R2 = {test_r2:.3f}, RMSE = {test_rmse:.3f}')
ax1.legend()
ax1.grid(True, alpha=0.3)

# 2. Feature Coefficients
ax2.barh(coef_df['Feature'], coef_df['Coefficient'], color=['red' if x < 0 else 'green' for x in coef_df
ax2.set_xlabel('Standardized Coefficient')
ax2.set_title('Ridge Regression Feature Coefficients')
ax2.axvline(x=0, color='black', linestyle='-', alpha=0.3)
ax2.grid(True, alpha=0.3)

# 3. Permutation Importance
ax3.barh(perm_df['Feature'], perm_df['Importance'], color='orange', alpha=0.7)
ax3.errorbar(perm_df['Importance'], range(len(perm_df)), xerr=perm_df['Std'], fmt='none', color='black',
ax3.set_xlabel('Permutation Importance')
ax3.set_title('Feature Importance (Permutation)')
ax3.grid(True, alpha=0.3)

# 4. Risk Tier Distribution
tier_pred_counts = pd.Series(y_pred_tiers).value_counts()
tier_comparison = pd.DataFrame({
    'Actual': tier_counts,
    'Predicted': tier_pred_counts
}).fillna(0)

tier_comparison.plot(kind='bar', ax=ax4, color=['lightcoral', 'lightblue'])
ax4.set_title('Risk Tier Distribution: Actual vs Predicted')
ax4.set_xlabel('Risk Tier')
ax4.set_ylabel('Number of Districts')
ax4.legend()
```

```
ax4.tick_params(axis='x', rotation=45)
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('graduation_rate_ridge_analysis.png', dpi=300, bbox_inches='tight')
plt.show()
```





```
=====
RIDGE REGRESSION MODEL SUMMARY
=====
```

**DATASET:**

Total districts: 196  
Features used: 5  
Train/Test split: 137/59

**RISK TIER THRESHOLDS:**

High Risk: < 0.830 (15 districts)  
Moderate Risk: 0.830 – 0.911 (29 districts)  
Low Risk: > 0.911 (15 districts)

```
print("=== RISK PREDICTION SYSTEM ===")
print("Practical implementation using trained Ridge regression model")
```

```
HIGH_RISK_THRESHOLD = p25_threshold
LOW_RISK_THRESHOLD = p75_threshold
```

```
class GraduationRiskPredictor:
    def __init__(self):
        self.feature_names = [
            'StudentToTeacherRatio',
            'Received_1003_Funds',
            'ELA_Performance',
            'Math_Performance',
            'Science_Performance'
        ]

        self.coefficients = {
            'StudentToTeacherRatio': 0.0175,
            'Received_1003_Funds': -0.0029,
            'ELA_Performance': 0.0177,
            'Math_Performance': 0.0202,
            'Science_Performance': -0.0135
        }
```

```
self.feature_means = {
    'StudentToTeacherRatio': 16.5,
    'Received_1003_Funds': 0.44,
    'ELA_Performance': 45.2,
    'Math_Performance': 35.8,
    'Science_Performance': 42.1
}

self.feature_stds = {
    'StudentToTeacherRatio': 3.2,
    'Received_1003_Funds': 0.50,
    'ELA_Performance': 12.8,
    'Math_Performance': 14.1,
    'Science_Performance': 13.5
}

self.intercept = 0.861 # Mean graduation rate from training
self.model_rmse = 0.064 # Model RMSE for confidence intervals

def standardize_features(self, features_dict):
    standardized = {}
    for feature in self.feature_names:
        if feature in features_dict and features_dict[feature] is not None:
            raw_value = features_dict[feature]
            mean = self.feature_means[feature]
            std = self.feature_stds[feature]
            standardized[feature] = (raw_value - mean) / std
        else:
            standardized[feature] = 0.0 # Mean imputation (standardized mean = 0)
    return standardized

def predict_graduation_rate(self, **features):
    std_features = self.standardize_features(features)

    prediction = self.intercept
    for feature in self.feature_names:
        prediction += self.coefficients[feature] * std_features[feature]

    prediction = max(0.0, min(1.0, prediction))
```

```

if prediction < HIGH_RISK_THRESHOLD:
    risk_category = "High Risk"
elif prediction <= LOW_RISK_THRESHOLD:
    risk_category = "Moderate Risk"
else:
    risk_category = "Low Risk"

ci_lower = max(0.0, prediction - 1.96 * self.model_rmse)
ci_upper = min(1.0, prediction + 1.96 * self.model_rmse)

return {
    'predicted_graduation_rate': prediction,
    'predicted_percentage': f"{prediction * 100:.1f}%",
    'risk_category': risk_category,
    'confidence_interval': f"{ci_lower*100:.1f}% - {ci_upper*100:.1f}%",
    'features_used': {k: v for k, v in features.items() if v is not None},
    'missing_features': [k for k, v in features.items() if v is None]
}

def explain_prediction(self, **features):
    std_features = self.standardize_features(features)

    print(f"\n{'='*60}")
    print("PREDICTION EXPLANATION")
    print(f"{'='*60}")

    feature_contributions = []
    for feature in self.feature_names:
        coef = self.coefficients[feature]
        std_val = std_features[feature]
        contribution = coef * std_val

        raw_val = features.get(feature, "Missing (imputed as mean)")

        feature_contributions.append({
            'feature': feature,
            'raw_value': raw_val,
            'contribution': contribution,

```

```

        'coefficient': coef
    })

feature_contributions.sort(key=lambda x: abs(x['contribution']), reverse=True)

print(f"\nFeature Contributions (sorted by impact):")
print(f"{'Feature':<25} {'Raw Value':<15} {'Contribution':<12} {'Coefficient':<12}")
print("-" * 70)

for fc in feature_contributions:
    contrib_str = f"{fc['contribution']:+.4f}"
    coef_str = f"{fc['coefficient']:+.4f}"
    raw_str = str(fc['raw_value'])[:14]
    print(f"{'fc['feature']':<25} {raw_str:<15} {contrib_str:<12} {coef_str:<12}")

base_rate = self.intercept
total_contribution = sum(fc['contribution'] for fc in feature_contributions)
final_prediction = base_rate + total_contribution

print(f"\nPrediction Calculation:")
print(f"  Base rate (intercept): {base_rate:.4f}")
print(f"  Feature contributions: {total_contribution:+.4f}")
print(f"  Final prediction: {final_prediction:.4f} ({final_prediction*100:.1f}%)")

predictor = GraduationRiskPredictor()

print(f"\nRisk Prediction System Initialized")
print(f"Model Features: {predictor.feature_names}")
print(f"Risk Thresholds: High < {HIGH_RISK_THRESHOLD:.1%}, Low > {LOW_RISK_THRESHOLD:.1%}")

print(f"\n" + "="*60)
print("EXAMPLE PREDICTIONS")
print("="*60)

print(f"\n1. HIGH-PERFORMING DISTRICT EXAMPLE:")
result1 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=14.5,
    Received_1003_Funds=0,
    ELA_Performance=65.2,

```

```
        Math_Performance=58.3,
        Science_Performance=62.1
    )

print(f"    Predicted Rate: {result1['predicted_percentage']}")
print(f"    Risk Category: {result1['risk_category']}")
print(f"    95% Confidence: {result1['confidence_interval']}")

print(f"\n2. AT-RISK DISTRICT EXAMPLE:")
result2 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=22.1,
    Received_1003_Funds=1,
    ELA_Performance=28.7,
    Math_Performance=19.4,
    Science_Performance=25.8
)

print(f"    Predicted Rate: {result2['predicted_percentage']}")
print(f"    Risk Category: {result2['risk_category']}")
print(f"    95% Confidence: {result2['confidence_interval']}")

print(f"\n3. MISSING DATA EXAMPLE:")
result3 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=16.8,
    Received_1003_Funds=0,
    ELA_Performance=42.1,
    Math_Performance=None,
    Science_Performance=None
)

print(f"    Predicted Rate: {result3['predicted_percentage']}")
print(f"    Risk Category: {result3['risk_category']}")
print(f"    Missing Features: {result3['missing_features']}")

predictor.explain_prediction(
    StudentToTeacherRatio=22.1,
    Received_1003_Funds=1,
    ELA_Performance=28.7,
    Math_Performance=19.4,
```

```
Science_Performance=25.8
)
```

=== RISK PREDICTION SYSTEM ===

Practical implementation using trained Ridge regression model

Risk Prediction System Initialized

Model Features: ['StudentToTeacherRatio', 'Received\_1003\_Funds', 'ELA\_Performance', 'Math\_Performance', 'Science\_Performance']

Risk Thresholds: High < 83.0%, Low > 91.1%

=====

EXAMPLE PREDICTIONS

=====

1. HIGH-PERFORMING DISTRICT EXAMPLE:

Predicted Rate: 89.3%

Risk Category: Moderate Risk

95% Confidence: 76.7% - 100.0%

2. AT-RISK DISTRICT EXAMPLE:

Predicted Rate: 85.8%

Risk Category: Moderate Risk

95% Confidence: 73.3% - 98.4%

3. MISSING DATA EXAMPLE:

Predicted Rate: 86.1%

Risk Category: Moderate Risk

Missing Features: ['Math\_Performance', 'Science\_Performance']

=====

PREDICTION EXPLANATION

=====

Feature Contributions (sorted by impact):

| Feature               | Raw Value | Contribution | Coefficient |
|-----------------------|-----------|--------------|-------------|
| StudentToTeacherRatio | 22.1      | +0.0306      | +0.0175     |
| Math_Performance      | 19.4      | -0.0235      | +0.0202     |
| ELA_Performance       | 28.7      | -0.0228      | +0.0177     |
| Science_Performance   | 25.8      | +0.0163      | -0.0135     |
| Received_1003_Funds   | 1         | -0.0032      | -0.0029     |

**Prediction Calculation:**

Base rate (intercept): 0.8610  
 Feature contributions: -0.0026  
 Final prediction: 0.8584 (85.8%)

```
print("=== RISK PREDICTION SYSTEM ===")
print("Practical implementation using trained Ridge regression model")

# Risk thresholds from our analysis (reuse from previous cells if available)
try:
    # Try to get thresholds from previous analysis
    HIGH_RISK_THRESHOLD = p25_threshold
    LOW_RISK_THRESHOLD = p75_threshold
    print(f"Using established thresholds: {HIGH_RISK_THRESHOLD:.3f} - {LOW_RISK_THRESHOLD:.3f}")
except:
    # Default thresholds based on our analysis
    HIGH_RISK_THRESHOLD = 0.830 # 25th percentile
    LOW_RISK_THRESHOLD = 0.911 # 75th percentile
    print(f"Using default thresholds: {HIGH_RISK_THRESHOLD:.3f} - {LOW_RISK_THRESHOLD:.3f}")

class GraduationRiskPredictor:
    """
    Practical risk prediction system for Washington State school districts.

    Takes district characteristics as input and returns:
    1. Predicted graduation rate
    2. Risk classification (High/Moderate/Low Risk)
    3. Confidence intervals and explanations
    """

    def __init__(self):
        # Initialize with trained model parameters (from our Ridge regression analysis)
        self.feature_names = [
            'StudentToTeacherRatio',
            'Received_1003_Funds',
            'ELA_Performance',
            'Math_Performance',
```

```
        'Science_Performance'
    ]

# Model coefficients from our trained Ridge regression ( $\alpha=10.0$ )
self.coefficients = {
    'StudentToTeacherRatio': 0.0175,
    'Received_1003_Funds': -0.0029,
    'ELA_Performance': 0.0177,
    'Math_Performance': 0.0202,
    'Science_Performance': -0.0135
}

# Feature means for standardization (from training data)
self.feature_means = {
    'StudentToTeacherRatio': 16.5,
    'Received_1003_Funds': 0.44,
    'ELA_Performance': 45.2,
    'Math_Performance': 35.8,
    'Science_Performance': 42.1
}

# Feature standard deviations for standardization
self.feature_stds = {
    'StudentToTeacherRatio': 3.2,
    'Received_1003_Funds': 0.50,
    'ELA_Performance': 12.8,
    'Math_Performance': 14.1,
    'Science_Performance': 13.5
}

# Model intercept and performance metrics
self.intercept = 0.861 # Mean graduation rate from training
self.model_rmse = 0.064 # Model RMSE for confidence intervals

def standardize_features(self, features_dict):
    """Standardize input features using training data statistics"""
    standardized = {}
    for feature in self.feature_names:
        if feature in features_dict and features_dict[feature] is not None:
```



```

        raw_value = features_dict[feature]
        mean = self.feature_means[feature]
        std = self.feature_stds[feature]
        standardized[feature] = (raw_value - mean) / std
    else:
        standardized[feature] = 0.0 # Mean imputation (standardized mean = 0)
    return standardized

def predict_graduation_rate(self, **features):
    """
    Predict graduation rate for a district given its characteristics.

    Parameters:
    -----
    StudentToTeacherRatio : float
        Student to teacher ratio (e.g., 15.2)
    Received_1003_Funds : int
        Whether district received federal 1003 funds (0 or 1)
    ELA_Performance : float
        Percentage meeting/exceeding ELA standards (e.g., 45.3)
    Math_Performance : float
        Percentage meeting/exceeding Math standards (e.g., 38.1)
    Science_Performance : float
        Percentage meeting/exceeding Science standards (e.g., 41.7)

    Returns:
    -----
    dict with prediction results
    """

    # Standardize features
    std_features = self.standardize_features(features)

    # Calculate prediction using Ridge regression formula
    prediction = self.intercept
    for feature in self.feature_names:
        prediction += self.coefficients[feature] * std_features[feature]

    # Ensure prediction is in valid range

```

```

prediction = max(0.0, min(1.0, prediction))

# Determine risk classification
if prediction < HIGH_RISK_THRESHOLD:
    risk_category = "High Risk"
    risk_color = "🔴"
elif prediction <= LOW_RISK_THRESHOLD:
    risk_category = "Moderate Risk"
    risk_color = "🟡"
else:
    risk_category = "Low Risk"
    risk_color = "🟢"

# Calculate confidence interval ( $\pm 1.96 \times \text{RMSE}$  for 95% CI)
ci_lower = max(0.0, prediction - 1.96 * self.model_rmse)
ci_upper = min(1.0, prediction + 1.96 * self.model_rmse)

return {
    'predicted_graduation_rate': prediction,
    'predicted_percentage': f"{prediction * 100:.1f}%",
    'risk_category': risk_category,
    'risk_emoji': risk_color,
    'confidence_interval': f"{ci_lower*100:.1f}% - {ci_upper*100:.1f}%",
    'features_used': {k: v for k, v in features.items() if v is not None},
    'missing_features': [k for k, v in features.items() if v is None]
}

def explain_prediction(self, **features):
    """Provide detailed explanation of prediction factors"""
    std_features = self.standardize_features(features)

    print(f"\n{'='*60}")
    print("PREDICTION EXPLANATION")
    print(f"{'='*60}")

    feature_contributions = []
    for feature in self.feature_names:
        coef = self.coefficients[feature]
        std_val = std_features[feature]

```

```

        contribution = coef * std_val

    # Get raw value for display
    raw_val = features.get(feature, "Missing (imputed as mean)")

    feature_contributions.append({
        'feature': feature,
        'raw_value': raw_val,
        'contribution': contribution,
        'coefficient': coef
    })

# Sort by absolute contribution
feature_contributions.sort(key=lambda x: abs(x['contribution']), reverse=True)

print(f"\nFeature Contributions (sorted by impact):")
print(f"{'Feature':<25} {'Raw Value':<15} {'Contribution':<12} {'Coefficient':<12}")
print("-" * 70)

for fc in feature_contributions:
    contrib_str = f"{fc['contribution']:+.4f}"
    coef_str = f"{fc['coefficient']:+.4f}"
    raw_str = str(fc['raw_value'])[:14]
    print(f"{fc['feature']:<25} {raw_str:<15} {contrib_str:<12} {coef_str:<12}")

base_rate = self.intercept
total_contribution = sum(fc['contribution'] for fc in feature_contributions)
final_prediction = base_rate + total_contribution

print(f"\nPrediction Calculation:")
print(f"  Base rate (intercept): {base_rate:.4f}")
print(f"  Feature contributions: {total_contribution:+.4f}")
print(f"  Final prediction: {final_prediction:.4f} ({final_prediction*100:.1f}%)")

# Initialize the predictor
predictor = GraduationRiskPredictor()

print(f"\n✅ Risk Prediction System Initialized")
print(f"Model Features: {predictor.feature_names}")

```

```
print(f"Risk Thresholds: High < {HIGH_RISK_THRESHOLD:.1%}, Low > {LOW_RISK_THRESHOLD:.1%}")

# Example usage with test cases
print(f"\n" + "="*60)
print("EXAMPLE PREDICTIONS")
print("="*60)

# Test Case 1: High-performing district
print(f"\n1. HIGH-PERFORMING DISTRICT EXAMPLE:")
result1 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=14.5,
    Received_1003_Funds=0,
    ELA_Performance=65.2,
    Math_Performance=58.3,
    Science_Performance=62.1
)

print(f"    Predicted Rate: {result1['predicted_percentage']}")
print(f"    Risk Category: {result1['risk_emoji']} {result1['risk_category']}")
print(f"    95% Confidence: {result1['confidence_interval']}")

# Test Case 2: At-risk district
print(f"\n2. AT-RISK DISTRICT EXAMPLE:")
result2 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=22.1,
    Received_1003_Funds=1,
    ELA_Performance=28.7,
    Math_Performance=19.4,
    Science_Performance=25.8
)

print(f"    Predicted Rate: {result2['predicted_percentage']}")
print(f"    Risk Category: {result2['risk_emoji']} {result2['risk_category']}")
print(f"    95% Confidence: {result2['confidence_interval']}")

# Test Case 3: Missing data example
print(f"\n3. MISSING DATA EXAMPLE:")
result3 = predictor.predict_graduation_rate(
    StudentToTeacherRatio=16.8,
```

```

    Received_1003_Funds=0,
    ELA_Performance=42.1,
    Math_Performance=None, # Missing data
    Science_Performance=None # Missing data
)

print(f"    Predicted Rate: {result3['predicted_percentage']}")
print(f"    Risk Category: {result3['risk_emoji']} {result3['risk_category']}")
print(f"    Missing Features: {result3['missing_features']}")

# Detailed explanation for one example
predictor.explain_prediction(
    StudentToTeacherRatio=22.1,
    Received_1003_Funds=1,
    ELA_Performance=28.7,
    Math_Performance=19.4,
    Science_Performance=25.8
)

print(f"\n" + "="*60)
print("USAGE INSTRUCTIONS")
print("="*60)
print(f"'''
To use this risk prediction system:

1. Call predictor.predict_graduation_rate() with district characteristics:

    result = predictor.predict_graduation_rate(
        StudentToTeacherRatio=15.5,
        Received_1003_Funds=0,
        ELA_Performance=45.2,
        Math_Performance=35.8,
        Science_Performance=42.1
    )

2. Access results:
    - result['predicted_percentage']: Graduation rate as percentage
    - result['risk_category']: High/Moderate/Low Risk classification
    - result['confidence_interval']: 95% confidence interval

```

```
3. For detailed explanations, use:
    predictor.explain_prediction(**same_parameters)
```

```
This system is ready for deployment in educational policy applications!
""")
print("="*60)
```

## ✓ Modeling Implementation #2: High-Performance Classification (Lead: Trevor)

```
!pip install lightgbm imbalanced-learn shap
```

```
Requirement already satisfied: lightgbm in /usr/local/lib/python3.12/dist-packages (4.6.0)
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dist-packages (0.14.0)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.48.0)
Requirement already satisfied: numpy>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from lightgbm) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lightgbm) (1.16.1)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.6.0)
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (1.4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from imbalanced-learn) (3.5.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.1)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (25.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.12.2)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2024.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2) (1.17.0)
```

```
# Complete data processing and modeling pipeline
import pandas as pd
```

```

import numpy as np
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE
import shap
import matplotlib.pyplot as plt
import seaborn as sns

# --- Data Preparation ---

grad_df = pd.read_csv('Report_Card_Graduation_2022-23_20250809.csv')
grad_filtered = grad_df[(grad_df['OrganizationLevel'] == 'District') & (grad_df['StudentGroup'] == 'All')]
district_data = grad_filtered[['DistrictName', 'GraduationRate']].copy()
district_data.dropna(subset=['GraduationRate'], inplace=True)
district_data['DistrictName'] = district_data['DistrictName'].str.lower().str.replace(' school district', '')

ratio_df = pd.read_csv('Report_Card_Student_Teacher_Ratio_School_Years_2017-18_to_2023-24.csv')
ratio_filtered = ratio_df[(ratio_df['SchoolYear'] == '2023-24') & (ratio_df['OrganizationLevel'] == 'LEA')]
ratio_data = ratio_filtered[['LEAName', 'StudentToTeacherRatio']].copy()
ratio_data.rename(columns={'LEAName': 'DistrictName'}, inplace=True)
ratio_data['DistrictName'] = ratio_data['DistrictName'].str.lower().str.replace(' school district', '')

funds_df = pd.read_csv('Report_Card_1003_Funds_2022-23_School_Year.csv')
funds_data = funds_df[['District Name']].copy()
funds_data.rename(columns={'District Name': 'DistrictName'}, inplace=True)
funds_data['Received_1003_Funds'] = 1
funds_data.drop_duplicates(inplace=True)
funds_data['DistrictName'] = funds_data['DistrictName'].str.lower().str.replace(' school district', '')

assessment_df = pd.read_csv('Report_Card_Assessment_Data_2022-23_School_Year_20250809.csv', low_memory=False)
assessment_filtered = assessment_df[(assessment_df['OrganizationLevel'] == 'District') & (assessment_df['Performance'] == 'Percent Consistent Grade Level Knowledge And Above')]
performance_col = 'Percent Consistent Grade Level Knowledge And Above'
assessment_filtered[performance_col] = assessment_filtered[performance_col].str.replace('%', '')
assessment_filtered[performance_col] = pd.to_numeric(assessment_filtered[performance_col], errors='coerce')

assessment_agg = assessment_filtered.pivot_table(
    index='DistrictName',

```

```

        columns='TestSubject',
        values=performance_col,
        aggfunc='mean'
    ).reset_index()

# Note: This rename is flexible; it will rename columns if they exist and ignore them if not.
assessment_agg.rename(columns={
    'ELA': 'ELA',
    'Math': 'Math',
    'Science': 'PctMetStandard_Science',
    'WIDAACC': 'WIDAACC'
}, inplace=True)

assessment_agg['DistrictName'] = assessment_agg['DistrictName'].str.lower().str.replace(' school district', '')

# --- Merge All Prepared DataFrames ---
final_df = pd.merge(district_data, ratio_data, on='DistrictName', how='left')
final_df = pd.merge(final_df, funds_data, on='DistrictName', how='left')
final_df['Received_1003_Funds'] = final_df['Received_1003_Funds'].fillna(0)
final_df = pd.merge(final_df, assessment_agg, on='DistrictName', how='left')

# Drop only the rows where GraduationRate or StudentToTeacherRatio is missing
final_df.dropna(subset=['GraduationRate', 'StudentToTeacherRatio'], inplace=True)

print(f"Created a merged dataset with {final_df.shape[0]} districts BEFORE imputation.")

# --- Build and Evaluate the Final Model ---

at_risk_threshold = final_df['GraduationRate'].quantile(0.25)
final_df['Is_At_Risk'] = (final_df['GraduationRate'] <= at_risk_threshold).astype(int)

# Add 'WIDAACC' to the list of features
features = [
    'StudentToTeacherRatio',
    'Received_1003_Funds',
    'ELA',
    'Math',

```



```
'PctMetStandard_Science',  
'WIDAACC'  
]  
target = 'Is_At_Risk'  
  
X = final_df[features]  
y = final_df[target]  
  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)  
  
# Impute missing values in ALL feature columns (including WIDAACC)  
imputer = SimpleImputer(strategy='mean')  
imputer.fit(X_train)  
X_train = imputer.transform(X_train)  
X_test = imputer.transform(X_test)  
  
# Apply SMOTE to handle imbalance  
smote = SMOTE(random_state=42)  
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)  
  
# Train the LightGBM model  
lgbm_clf = lgb.LGBMClassifier(random_state=42)  
lgbm_clf.fit(X_train_resampled, y_train_resampled)
```





[illegible]

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

▼ LGBMClassifier ⓘ

```
LGBMClassifier(random_state=42)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have va
warnings.warn(
```

```
--- FINAL CLASSIFICATION REPORT (WITH WIDAACC FEATURE) ---
```

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Not At-Risk (0) | 0.87      | 0.87   | 0.87     | 30      |
| At-Risk (1)     | 0.60      | 0.60   | 0.60     | 10      |

✓ **Step 7: Model Interpretation with SHAP**

|              |      |    |
|--------------|------|----|
| accuracy     | 0.80 | 40 |
| macro avg    | 0.73 | 40 |
| weighted avg | 0.80 | 40 |

```
print("\n--- Initializing SHAP Analysis ---")
```

```
# 1. Create a SHAP explainer object
```

```
explainer = shap.TreeExplainer(lgbm_clf)
```

```
# 2. Calculate SHAP values for the test set
```

```
shap_values = explainer.shap_values(X_test)
```

```
# 3. Get feature names for plotting
```

```
feature_names = [
    'StudentToTeacherRatio',
    'Received_1003_Funds',
    'ELA',
    'Math',
    'PctMetStandard_Science',
    'WIDAACC'
]
```

```
X_test_df = pd.DataFrame(X_test, columns=feature_names)
```

```
# 4. Generate and display the SHAP plots
```

```
# Plot 1: SHAP Feature Importance (Bar Plot)
```

```
print("\n--- SHAP Feature Importance Plot ---")
```

```
shap.summary_plot(shap_values, X_test_df, plot_type="bar", show=False)
```

```
plt.title("SHAP Feature Importance for 'At-Risk' Class")
plt.tight_layout()
plt.savefig('shap_feature_importance.png', bbox_inches='tight')
# THE FIX: Use plt.show() to display the plot inline
plt.show()
print("Saved SHAP feature importance plot to 'shap_feature_importance.png'")

# Plot 2: SHAP Summary Plot (Beeswarm)
print("\n--- SHAP Summary Plot ---")
shap.summary_plot(shap_values, X_test_df, show=False)
plt.title("SHAP Summary Plot for 'At-Risk' Class")
plt.tight_layout()
plt.savefig('shap_summary_plot.png', bbox_inches='tight')
# THE FIX: Use plt.show() to display the plot inline
plt.show()
print("Saved SHAP summary plot to 'shap_summary_plot.png'")

print("\nSHAP analysis complete.")
```





## Strategic Model Trade-offs: Interpretability vs. Predictive Power

--- Initializing SHAP Analysis ---  
 --- SHAP Feature Importance Plot ---  
 /usr/local/lib/python3.12/dist-packages/shap/explainers/\_tree.py:583: UserWarning: LightGBM binary classi-  
 warning: warn

### Ridge Regression vs. LightGBM: When to Choose Which Approach

Our analysis implemented two distinct modeling approaches, each with strategic advantages for different stakeholder needs:

#### Ridge Regression Advantages:

- **Transparency:** Coefficients directly show how each factor influences graduation rates
- **Stakeholder Communication:** Education officials can easily explain predictions to school boards, parents, and policymakers
- **Regulatory Compliance:** Many education policies require explainable decision-making processes